

Technical Report

Real-Time Visualization of Digital Modulation and Channel Impairments Using a Single PlutoSDR

Based on repository review of HowardWHSrun/E78FinalProject, including source code, README, and accompanying presentation materials

Scope of this report. This document explains the architecture, digital signal processing chain, software design, synchronization strategy, GUI behavior, experimental modes, strengths, limitations, and recommended improvements of the project. Performance numbers reported in the accompanying presentation are included, but they are clearly distinguished from code-confirmed behavior when they were not independently reproduced.

Repository	HowardWHSrun/E78FinalProject
Primary stack	Python, NumPy, SciPy, PyQt6, PyQtGraph, pyadi-iio, ADALM-PLUTO SDR
Operating modes	Hardware loopback and software simulation fallback
Prepared	17 April 2026

Repository URL: <https://github.com/HowardWHSrun/E78FinalProject>

Executive Summary

E78FinalProject is a teaching-oriented digital communications workbench that combines a software-defined radio front end, a framed digital baseband waveform, software-injected channel impairments, receiver synchronization, hard-decision demodulation, and a real-time visualization interface. The project is notable because it does not stop at simulation: it is designed to run through a single ADALM-PLUTO SDR in hardware loopback, while also offering a software-only fallback for portability and debugging.

Architecturally, the project separates signal generation and reception from rendering. A background QThread receives data from the PlutoSDR (or a software loopback), applies impairments, runs synchronization and demodulation, computes metrics, and writes the most recent results into shared attributes protected by a mutex. The PyQt6 GUI then polls those results at a fixed timer interval of 50 ms, which yields a stable user experience and avoids the event-queue backlog that would arise if every processed buffer were emitted as an individual GUI signal.

From a digital communications standpoint, the implemented waveform is straightforward and well chosen for pedagogy. It supports four uncoded schemes - BPSK, QPSK, 8-PSK, and 16-QAM - uses root-raised-cosine pulse shaping with roll-off factor 0.35 and 8 samples per symbol, prepends a repeated Barker-13 preamble for synchronization, and performs nearest-neighbor hard-decision demodulation after matched filtering and timing recovery.

The strongest aspect of the project is its ability to connect theory to intuition. Students can observe how additive noise lifts the spectrum floor, how phase offset rotates a constellation, how frequency offset induces spiral motion before correction, and how multipath closes an eye diagram. The BER sweep mode extends this from visual intuition to quantitative validation by tracing measured BER against closed-form reference curves.

The code also reveals several technically important limitations. The impairment models are intentionally simple, the synchronization chain is relatively lightweight, there is no adaptive equalization, and the throughput metric counts all decoded bits after the first detected preamble in a buffer rather than enforcing an exact one-frame payload boundary. As a result, the throughput visualization is useful pedagogically but should not be interpreted as a rigorous link-layer throughput benchmark without further refinement.

1. Project Goals, Context, and Scope

The project addresses a common pedagogical gap in undergraduate digital communications: equations such as BER formulas, carrier offset models, and eye-diagram interpretations are often learned symbolically but not experienced dynamically. The stated goal of the project is to let a user manipulate channel conditions interactively and see, in real time, how the signal degrades and how the receiver responds.

The repository and presentation materials show that the scope includes both a live interactive mode and a BER sweep mode. In live mode, the user can switch modulation schemes and

manipulate impairments with sliders. In BER sweep mode, the project automatically measures BER across an SNR range for each supported modulation and overlays those measurements with theoretical curves.

This report focuses on five questions: what the system does, how the end-to-end DSP chain is implemented, how the software is structured for responsive real-time visualization, what engineering choices are especially strong, and which parts of the design should be revised if the project is extended into a more robust communications testbed.

Important interpretation note. This report distinguishes between code-confirmed behavior and presentation-reported outcomes. When a number such as frame rate, implementation loss, or lock time appears in the accompanying presentation but is not directly reproducible from source inspection alone, the report treats it as a reported claim rather than a verified benchmark.

2. Evidence Base and Review Method

The analysis underlying this report was based on the repository README, the Python source files `main.py`, `gui.py`, `dsp_worker.py`, `pluto_interface.py`, `modulation.py`, `impairments.py`, `synchronization.py`, `ber_sweep.py`, and the accompanying presentation and script. The code review was used to reconstruct the end-to-end architecture and identify data flow, synchronization logic, metric computation, and threading design.

Source	What it contributes to the report
README.md	Project overview, supported impairments, hardware setup, installation, and high-level file map
main.py	Application entry point and GUI launch behavior
gui.py	Widget structure, plotting behavior, live mode, sweep mode, slider ranges, and readout logic
dsp_worker.py	Live receive-impaire-sync-demod pipeline and shared-memory polling model
pluto_interface.py	Hardware connection behavior, Pluto defaults, simulation fallback, transmit scaling, receive buffering

modulation.py	Constellation maps, pulse shaping, matched filtering, hard-decision demodulation, BER and EVM calculations
impairments.py	AWGN, phase, frequency, multipath, and flat Rician fading models
synchronization.py	Barker preamble, frame detection, coarse CFO estimation, phase estimation, Gardner timing recovery
ber_sweep.py	Automated BER-vs-SNR evaluation and theoretical reference formulas
presentation.tex and script.md	Project framing, claimed results, lessons learned, and some qualitative descriptions

3. System Overview

At the highest level, the application is a framed baseband transmitter and receiver wrapped inside a real-time GUI. Random payload bits are mapped to complex constellation symbols, pulse shaped, prepended with a synchronization preamble, cyclically transmitted through a PlutoSDR or software loopback, passed through a controllable impairment stage, synchronized and demodulated, and then displayed through multiple live visualizations.

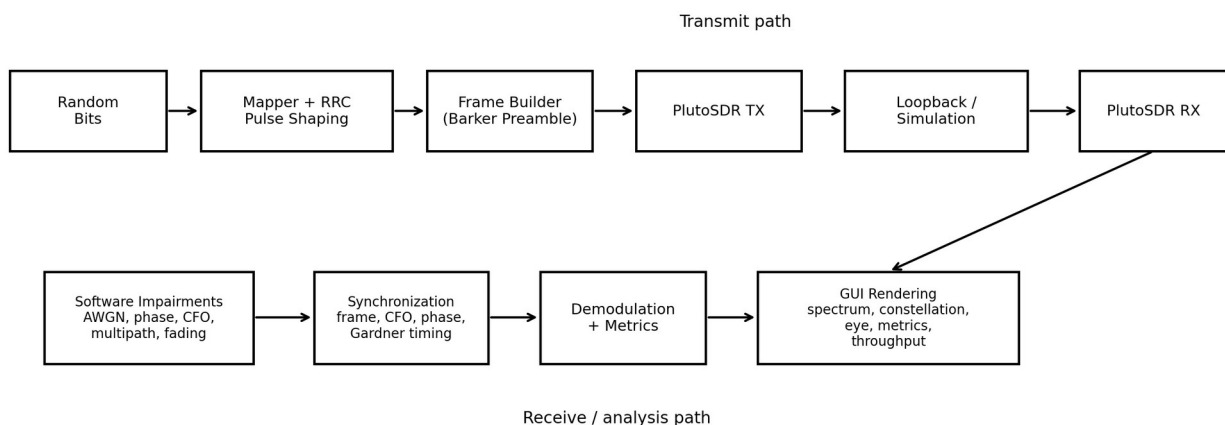


Figure 1. Reconstructed high-level architecture of the project based on the repository source code.

A notable design choice is that software impairments are applied after reception rather than being baked into the transmitted waveform. This reduces UI latency because moving a slider does not require the application to rebuild the transmit buffer. The worker can instead apply the

impairment chain directly to each received buffer and immediately re-run synchronization and demodulation.

4. Operating Modes, Requirements, and Default Parameters

The project supports two explicit operating modes. In hardware mode, it attempts to connect to a real PlutoSDR, configure RF parameters, stream a cyclic transmit buffer, and receive data back through the Pluto receive path. In simulation mode, it falls back to a software loopback that tiles the transmit waveform and adds low-level complex noise so the remainder of the DSP and visualization pipeline can still run.

Parameter	Default value	Role in the system
Local oscillator	915 MHz	Sets Pluto TX and RX center frequency in hardware mode
Sample rate	1 MSPS	Defines the sample clock for transmit, receive, and impairment frequency scaling
RF bandwidth	1 MHz	Sets Pluto transmit and receive analog bandwidths
TX gain	-30 dB	Reduces transmit power for safe hardware loopback operation
RX gain mode	slow_attack	Lets Pluto adjust RX gain automatically
Buffer size	2^{14} samples	Receive buffer length and software loopback block size
Samples per symbol	8	Baseband oversampling factor for pulse shaping and timing recovery
RRC roll-off	0.35	Pulse-shaping excess bandwidth
RRC taps	101	Transmit and matched-filter

		impulse-response length
Payload bits	2048	Nominal payload size generated by the worker for each frame

The README additionally recommends a loopback cable between Pluto TX and RX and a 30 dB attenuator on the receive side to protect the front end. That recommendation is consistent with good laboratory practice for direct RF loopback experiments.

5. Digital Waveform and Frame Structure

The transmitted waveform is organized around discrete payload bits and framed complex symbols. For each selected modulation scheme, the payload is mapped into constellation points, prepended with a known preamble, upsampled by a factor of eight, and pulse shaped with a root-raised-cosine filter. The Pluto then transmits the resulting complex IQ waveform as a cyclic buffer.

Scheme	Bits per symbol	Constellation type	Primary pedagogical use
BPSK	1	Two antipodal points	Most noise-robust baseline for BER comparisons
QPSK	2	Four equally spaced phase states	Balanced example of spectral efficiency and robustness
8-PSK	3	Eight phase states on a circle	Illustrates reduced noise margin as angular spacing shrinks
16-QAM	4	4x4 rectangular constellation	Shows high spectral efficiency and strong sensitivity to distortion

Reference Constellations Used in the Project

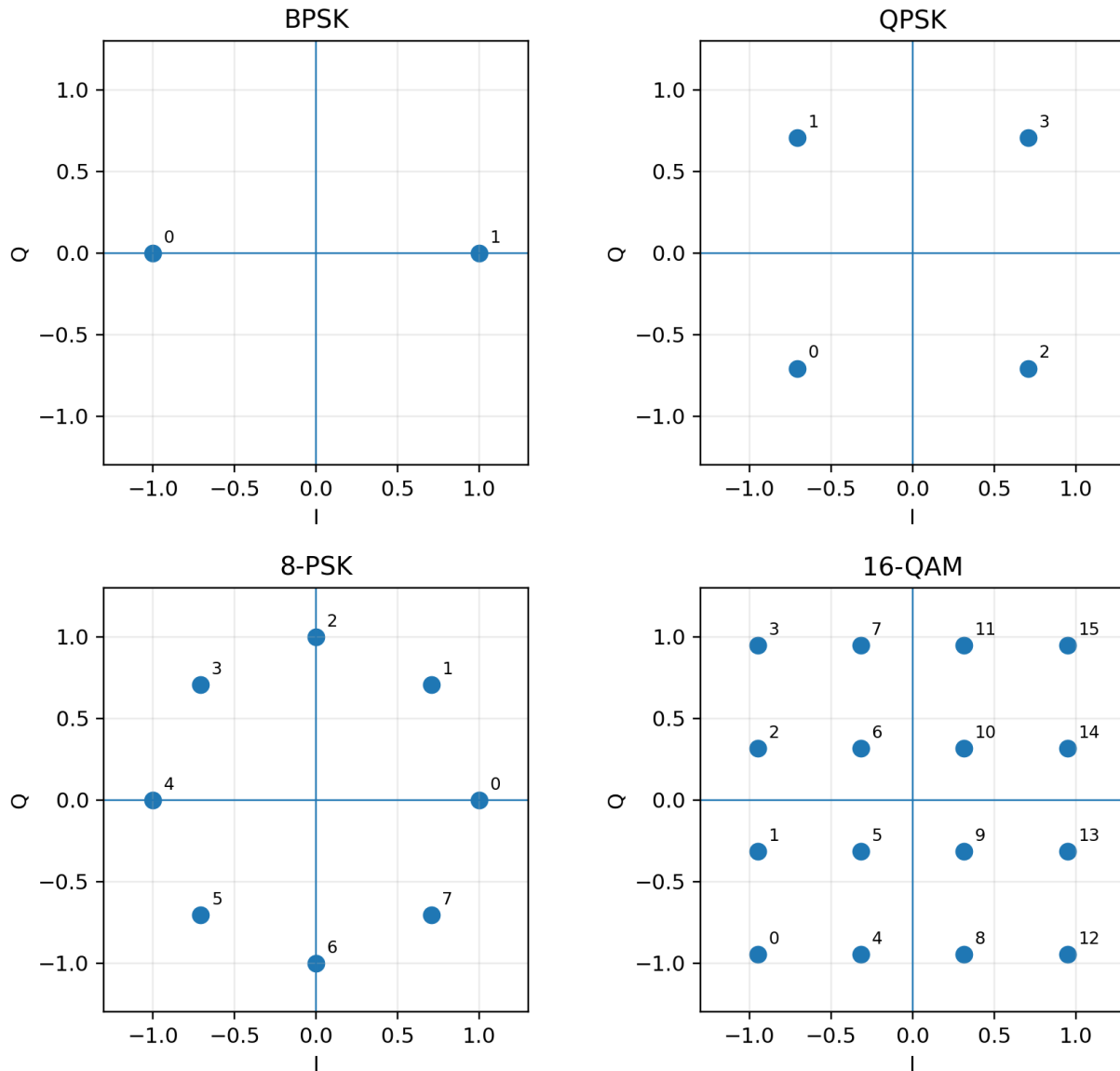


Figure 2. Reference constellation families corresponding to the schemes implemented in modulation.py.

The preamble is a Barker-13 sequence repeated four times, which yields 52 known preamble symbols. In code, the frame structure is simply [preamble | payload symbols]. This point matters because the presentation script mentions pilot symbols, but the reviewed source code does not append a separate pilot block after the Barker preamble.

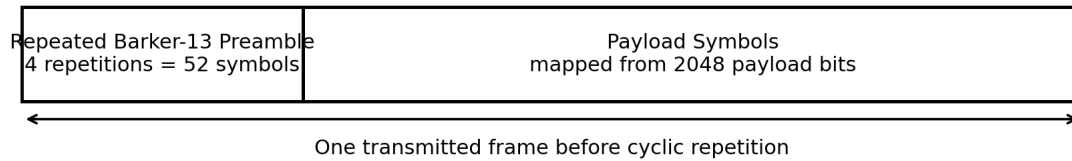


Figure 3. Frame structure reconstructed from `synchronization.py` and `dsp_worker.py`.

Code/presentation discrepancy. The source code builds each frame as preamble plus payload only. The accompanying script states that the frame contains preamble, pilot symbols, and payload. If a separate pilot field was used during development, it is not present in the current repository revision and the written materials should be updated to match the actual implementation.

5.1 Pulse shaping and matched filtering

The project uses root-raised-cosine pulse shaping with $\alpha = 0.35$, 101 taps, and 8 samples per symbol. The filter is normalized so that transmit filtering followed by matched filtering approximates a raised-cosine overall response. This is a sound choice for teaching because the spectrum and eye diagram both become meaningful: the spectrum reflects excess bandwidth, while the eye opening reflects timing margin after matched filtering.

On the transmit side, symbols are inserted into an upsampled array at positions spaced by the samples-per-symbol factor and then convolved with the RRC taps. On the receive side, the same taps are used for matched filtering. The implementation therefore preserves the familiar textbook structure of pulse shaping, matched filtering, and symbol-rate recovery.

5.2 Modulation and demodulation details

Constellation maps are precomputed explicitly for each supported scheme. QPSK and 16-QAM are Gray coded in the source. Demodulation uses a vectorized nearest-neighbor rule: each received complex symbol is compared against all ideal reference points and is assigned the bit label of the closest point. This is the correct maximum-likelihood hard-decision detector for AWGN when all symbols are equiprobable and the receiver has already corrected bulk phase, timing, and frequency impairments.

The code caches the ideal point array and bit labels for each scheme, which avoids rebuilding demodulation lookup data on every call. That is a modest but appropriate optimization for a real-time educational application.

6. Channel Impairment Engine

The impairment module is intentionally simple and readable. Each impairment acts on a complex IQ array and returns a modified copy, allowing the impairments to be chained in a predictable

order. The wrapper applies them as multipath, fading, frequency offset, phase offset, and then AWGN. That order is physically reasonable for a pedagogical channel model: deterministic or multiplicative channel effects are imposed before additive noise.

Impairment	Implementation concept	GUI control range	Expected visible effect
AWGN	Adds complex Gaussian noise at target SNR relative to signal power	0 to 40 dB	Noise floor rises, constellations spread, eye closes, BER increases
Phase offset	Multiplies by $\exp(j\phi)$	-180 to +180 deg	Constellation rotates rigidly
Frequency offset	Applies linear phase ramp across samples	-5000 to +5000 Hz	Spectrum shifts and constellation may appear to spin before correction
Multipath	Adds delayed and scaled copy of waveform	0 to 10 symbol periods, amplitude 0 to 1	Eye closure and inter-symbol interference
Flat fading	Applies samplewise Rician coefficient	K = -10 to 40 dB	Amplitude and phase scatter increase as the diffuse component grows

From an engineering perspective, the impairment library is adequate for visualization but not yet a full channel emulator. Multipath is limited to a single delayed echo, fading is flat rather than frequency selective, and AWGN is referenced to instantaneous signal power rather than to a full link-budget model. Those simplifications are acceptable for this project's objective, but they define the boundary between a teaching demonstrator and a communications research platform.

7. Receiver Synchronization

Synchronization is the most technically significant part of the receiver. Without it, the visualizations and BER estimates would be misleading because the receiver would not know where frames start, how much carrier offset is present, or when to sample each symbol.

Stage	What the code does	Why it matters
Frame detection	Cross-correlates received samples against a pulse-shaped version of the repeated Barker preamble	Finds the start of the frame
Coarse frequency offset estimation	Uses the phase difference between adjacent repeated Barker blocks	Removes large carrier offset before symbol decisions
Matched filtering	Convolve aligned samples with the same RRC taps used at TX	Improves SNR and restores pulse shape
Timing recovery	Runs Gardner TED with a first-order loop and linear interpolation	Finds good symbol sampling instants
Phase estimation	Computes the residual phase from the recovered preamble symbols	Rotates symbols toward the correct reference constellation
Payload extraction	Splits corrected symbols into preamble symbols and all following symbols	Feeds demodulation and metric computation

The synchronization design is both practical and understandable. A repeated Barker preamble is a strong educational choice because it supports frame detection and coarse carrier-offset estimation without requiring a complex acquisition algorithm. Likewise, Gardner timing recovery is one of the classic symbol-timing methods students are expected to recognize.

At the same time, the current synchronization chain is deliberately lightweight. The frequency estimator uses only the first two repeated Barker blocks. Timing recovery uses a first-order loop with linear interpolation rather than a higher-order loop with a dedicated interpolation filter. There is no adaptive equalizer after matched filtering, so severe multipath or dispersive distortion cannot be fully undone. These are acceptable tradeoffs in a real-time visualization project, but they place a ceiling on robustness.

8. Software Architecture and Concurrency Model

The software is split across focused modules, and the separation is generally strong. `main.py` only launches the GUI. `gui.py` defines widgets, plots, mode controls, and periodic rendering. `dsp_worker.py` performs the live receive-process-display pipeline. `pluto_interface.py` manages the SDR or simulation backend. `modulation.py`, `impairments.py`, `synchronization.py`, and `ber_sweep.py` keep the core DSP logic outside the GUI layer.

Module	Primary responsibility	Architectural value
<code>main.py</code>	Create QApplication and show the main window	Keeps program launch simple
<code>gui.py</code>	Owens all widgets, plots, mode selection, and rendering updates	Separates presentation layer from DSP
<code>dsp_worker.py</code>	Runs live processing in a QThread and stores latest results	Prevents the GUI from blocking on DSP
<code>pluto_interface.py</code>	Handles SDR connection, TX streaming, RX acquisition, and simulation fallback	Encapsulates hardware-specific logic
<code>modulation.py</code>	Mapping, pulse shaping, matched filtering, demodulation, BER, EVM	Concentrates signal-space operations
<code>impairments.py</code>	Applies channel distortions	Makes live channel control easy to understand and extend
<code>synchronization.py</code>	Preamble-based recovery and timing logic	Centralizes receiver acquisition and alignment
<code>ber_sweep.py</code>	Runs automated BER experiments and theory overlay	Separates live mode from offline-style evaluation

The most important software architecture decision is the choice to use a shared latest-result model instead of emitting heavy per-buffer GUI updates. The worker writes the newest processed arrays

and metrics into member variables protected by a mutex. The GUI uses a timer that fires every 50 ms and reads whatever the latest available snapshot is. This means the GUI always renders current information without accumulating a backlog of stale updates. For real-time educational visualization, this is exactly the right design choice.

Why this matters. If the worker emitted a full Qt signal with large arrays for every received buffer, the event queue could grow faster than the GUI can draw, causing lag and eventual unresponsiveness. The shared-memory polling model bounds GUI workload to the display rate rather than the SDR processing rate.

9. GUI Design and Visualization Strategy

The interface supports two modes: live visualization and BER sweep. In live mode, the user sees a power spectrum, constellation diagram, eye diagram, rolling BER/EVM/SNR plot, and a throughput plus cumulative-bits panel. The GUI also provides descriptive information popups for each plot type, which increases the pedagogical value because the software explains what the user should look for.

The plots are well chosen. The power spectrum shows spectral occupancy and noise floor changes. The constellation captures symbol-space degradation. The eye diagram reveals inter-symbol interference, noise, and timing uncertainty. The metrics plot turns qualitative behavior into time histories. The throughput panel adds a sense of data flow and receiver lock stability.

One subtle implementation detail is that the program auto-starts shortly after launch. If hardware is unavailable, it falls back to simulation and still starts live operation. This is excellent for demonstration settings because the application comes alive immediately, although it differs slightly from the README's more manual run description.

10. Metrics and Their Computation

The project computes four major live metrics: BER, EVM, estimated SNR, and throughput. BER is computed by comparing transmitted payload bits against the recovered bit stream, truncated to the shorter of the two arrays. EVM is computed as the RMS symbol error magnitude divided by RMS reference magnitude, expressed as a percentage. The estimated SNR is computed from the power ratio between the ideal symbols and the error vectors between received and ideal symbols.

These metric definitions are sensible for visualization. BER is the ultimate communications performance measure, EVM links waveform quality to standard digital-modulation practice, and the estimated SNR gives users a more intuitive scale for impairment severity. Together they create a helpful bridge between geometry in the constellation plane and numerical link quality.

10.1 Throughput metric: useful but not exact

The throughput label deserves special comment. The GUI computes instantaneous throughput from the number of decoded bits returned by the worker divided by elapsed time between worker updates. However, the synchronization function returns all symbols from the first detected preamble to the end of the current receive buffer, and the worker sets `bits_decoded` equal to the length of the resulting demodulated stream. In other words, the code does not explicitly enforce a single exact payload block per buffer before computing throughput.

That means the throughput plot is a meaningful relative indicator of how much decodable information is flowing through the receiver, but it is not yet a strict link-layer throughput measurement tied to one precisely delimited frame. This also helps explain why the presentation's quoted throughput scales roughly with modulation order: higher-order modulations compress more payload bits into fewer symbols, so more decodable bits may fit after the first preamble within a fixed receive buffer.

Practical recommendation. For a rigorous throughput metric, the worker should detect explicit frame boundaries after synchronization, decode exactly one or a counted integer number of complete payload frames, and compute throughput from known payload lengths rather than from the entire post-preamble tail of the buffer.

11. BER Sweep Mode

BER sweep mode turns the project from an interactive demonstrator into a measurement instrument. The sweep worker iterates over all four modulation schemes and over a user-selected SNR range. At each SNR point it generates payload bits, maps them to symbols, constructs a frame, pulse-shapes the waveform, optionally transmits it through the Pluto hardware, adds AWGN, re-runs synchronization and demodulation, and records the measured BER.

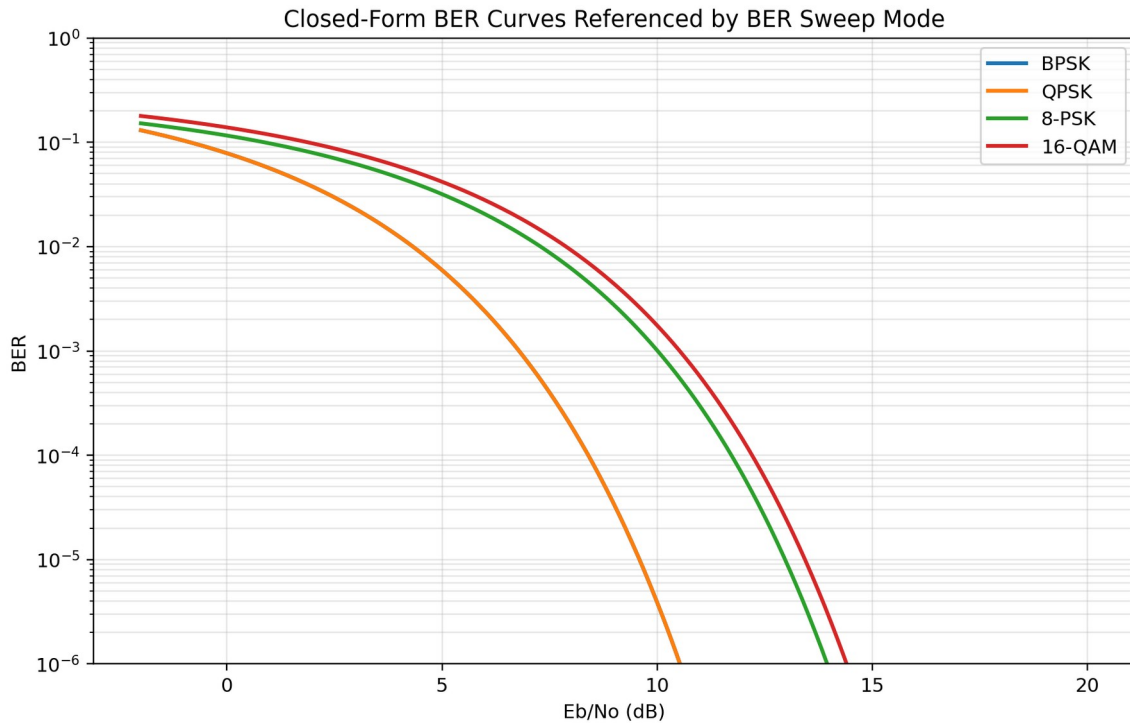


Figure 4. Closed-form BER curves that correspond to the formulas implemented in `ber_sweep.py`.

Theoretical reference curves are included for BPSK, QPSK, 8-PSK, and 16-QAM. The code converts SNR to E_b/N_0 using the standard relation $E_b/N_0 = E_s/N_0 - 10 \log_{10}(k)$, where k is bits per symbol. This is conceptually sound and makes the plot easier to compare against textbook curves.

An important scope note is that the sweep injects AWGN only. The live-mode impairment sliders do not feed into sweep mode. That is a reasonable decision because theoretical closed-form BER comparisons are most meaningful under AWGN, but it should be documented clearly so that users do not assume sweep mode validates performance under multipath or fading.

12. Hardware Interface and Deployment Behavior

The Pluto interface wrapper attempts to auto-detect a PlutoSDR through `libiio` context scanning and falls back to the default Pluto IP if needed. On successful connection it configures the transmit and receive local oscillators, sample rate, RF bandwidth, gain mode, and buffer settings. When transmitting, it scales the waveform to the Pluto DAC range by normalizing the peak magnitude and multiplying by 2^{14} .

This part of the code shows a practical understanding of lab deployment. The interface also catches connection or streaming failures and converts them into a custom `HardwareError`, which lets the GUI stop cleanly and prompt the user to reconnect or use simulation mode. That behavior makes the application more reliable in real demonstrations where USB devices may disconnect unexpectedly.

13. Technical Strengths

1. The project unifies hardware, DSP, visualization, and measurement in a single coherent application rather than presenting them as separate scripts.
2. The modulation and impairment modules are readable enough for teaching, which makes the codebase itself a learning artifact.
3. The repeated Barker preamble plus Gardner timing structure is technically meaningful and faithful to core communications concepts.
4. The shared-memory worker-to-GUI architecture is an excellent real-time software design decision.
5. The software-only fallback significantly increases portability and makes the project demonstrable even when hardware or drivers are unavailable.
6. The BER sweep mode adds quantitative validation on top of qualitative visualization, which greatly strengthens the educational value.

14. Limitations, Risks, and Code-Level Improvement Opportunities

Issue	Why it matters	Recommended fix
No explicit post-sync frame length enforcement	BER and throughput are computed from all post-preamble symbols in a buffer, which can blur exact frame accounting	Decode complete frames only and trim extra samples or symbols
No equalization	Simple multipath may be visible, but the receiver cannot meaningfully recover from stronger ISI	Add an LMS or decision-directed equalizer after timing recovery
Simple channel models	Two-tap multipath and flat fading are pedagogically useful but limited	Add multipath channel impulse response support and frequency-selective fading
Lightweight CFO and timing loops	Robustness may degrade under strong impairments or large offsets	Add loop filter tuning, pilotless carrier tracking, and better interpolation
Limited automated validation	The repository has no	Add test vectors for

	dedicated unit tests or regression harness	modulation, synchronization, and BER formulas
README and script drift	Documentation inconsistencies reduce trust and make grading or reuse harder	Update written materials to match the current code

Beyond those structural issues, there are smaller points worth noting. The simulation loopback always adds a small background noise term before any live-mode AWGN slider is applied. The GUI's descriptive material refers to typical throughput values by modulation order, but because throughput is based on decoded post-preamble tail length, those numbers should be interpreted as empirical display behavior rather than as a formally defined PHY data rate.

15. Documentation and Internal Consistency Review

The repository is generally well organized, but the documentation should be revised in several places to stay fully consistent with the code.

- The README describes four live plot panels, while gui.py clearly implements a fifth throughput and cumulative-bits panel.
- The presentation script mentions pilot symbols in the frame, but synchronization.py and dsp_worker.py only implement a repeated Barker preamble plus payload.
- The README gives a generic 'Click Connect' instruction, whereas the GUI exposes separate 'Connect Hardware' and 'Simulation Mode' buttons and also auto-starts on launch.
- The presentation claims lock time and implementation-loss figures that are plausible but are not directly validated inside the repository through a reproducible benchmark script.

16. Reported Performance Claims and Interpretation

The presentation materials report that measured BER curves match theory within less than 1 dB for all four schemes, that hardware loopback adds roughly 0.5 to 1 dB of loss relative to simulation, that simulation mode runs at approximately 20 FPS with about 6 ms DSP time per frame, and that hardware mode is somewhat slower because of USB latency. Those claims are plausible given the design, but they should be treated as presentation-reported results unless accompanied by saved datasets or benchmark scripts.

The presentation also reports sustained decoded throughput of roughly 40 kbit/s for QPSK and 80 kbit/s for 16-QAM, with more than 1 Mbit accumulated in 30 seconds. Given the worker's bits_decoded logic, those displayed values are believable as a real-time display statistic. However, they should not yet be presented as exact payload throughput without first tightening frame accounting.

Best way to strengthen the reported-results section. Add a reproducible experiment script that saves measured BER points, synchronization success rates, estimated carrier offsets, and timing-lock behavior to disk. That would let the report cite measured results from the repository itself rather than relying on presentation slides.

17. Recommended Next Steps

- Enforce explicit frame boundaries after synchronization so BER and throughput are computed on exact payloads.
- Add optional pilot symbols or a separate training section if improved carrier and timing tracking is desired.
- Implement a simple adaptive equalizer to extend the educational value under multipath and mild dispersive channels.
- Add a logging mode that saves IQ snapshots and metrics so students can compare offline analysis to live behavior.
- Create automated tests for constellation mapping, RRC filter normalization, Barker detection, BER formulas, and Gardner recovery under synthetic impairments.
- Refresh README and presentation text so the frame structure, plot count, and user workflow all match the current revision.

18. Conclusion

E78FinalProject is a strong course project because it integrates concepts that are often taught separately: modulation, pulse shaping, synchronization, receiver metrics, SDR hardware control, and real-time visualization. The project demonstrates mature engineering judgment in several places, especially in the choice of a simulation fallback, the modular source layout, and the worker-thread design that decouples DSP from GUI rendering.

Its main weaknesses are not conceptual but developmental: frame accounting should be tightened, the documentation should be synchronized with the code, and more rigorous measurement scripts would make the project's performance claims easier to defend in a formal report or paper. Even with those limitations, the project succeeds in its central mission. It makes digital communications visible, interactive, and experimentally tangible.

Appendix A. Reproduction Checklist

- Install Python dependencies from requirements.txt.
- Connect Pluto TX to RX through an attenuated loopback path if using hardware.
- Launch the application with python main.py.
- Verify live mode first in simulation if Pluto drivers are unavailable.
- For BER sweep mode, keep impairments conceptually limited to AWGN because the theoretical overlay is derived for uncoded AWGN behavior.

Appendix B. File-by-File Engineering Notes

`main.py` is intentionally minimal, which is appropriate. `gui.py` is the largest orchestration layer and contains most user-facing behavior. `dsp_worker.py` contains the live processing logic and should be the first place to modify if the receiver or metrics are extended. `synchronization.py` is the most technically sensitive DSP component and the most likely source of future tuning work if the project is pushed to harsher channels or looser hardware conditions.

If the project were advanced into a senior thesis, research demo, or publishable educational tool, the natural development path would be: first improve frame accounting, second add saved experiment outputs, third strengthen channel and synchronization models, and finally package the application with reproducible test cases and an updated manual.